Python for RAG Pipelines

A Practical Developer Reference Guide to Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) is a powerful architecture that augments LLM capabilities by bringing external authoritative documentation into the prompt context dynamically. Python handles the orchestration, vector calculations, and pipeline connectivity natively.

> **The 4 Pillars of RAG Architecture:**
>
> 1. **Ingestion:** Parsing raw text strings out of corporate data formats (PDFs, Markdown, Web pages).
> 2. **Chunking:** Segmenting data structurally into token-friendly context blocks.
> 3. **Embedding & Indexing:** Generating statistical vectors representing data semantics.
> 4. **Synthesis:** Injecting targeted coordinates directly into language model prompts.

1. Core Python RAG Libraries Matrix

CATEGORY	LIBRARIES	ROLE IN PIPELINE
Document Parsers	`PyPDF2`, `pdfplumber`, `fitz`	Reading, scanning, and extracting strings from document assets.
Text Splitters	`langchain-text-splitters`	Parsing chunks gracefully while ensuring semantic paragraphs remain together.
Vector Store	`chromadb`, `faiss-cpu`	Localized in-memory or disk-backed databases optimized for semantic retrieval.
Orchestration Frameworks	`langchain`, `llamaindex`	Ready-made abstractions to stitch vector indexes, pipelines, and prompt variables together.
Model Clients	`openai`, `ollama`	Interfacing with third-party APIs or running local execution layers.

2. Python Code Blueprint

Step 1 & 2: Document Parsing & Chunking

```python
from PyPDF2 import PdfReader
from langchain.text_splitter import RecursiveCharacterTextSplitter

# 1. Read PDF Pages
reader = PdfReader("manual.pdf")
text = "".join([page.extract_text() for page in reader.pages])

# 2. Slice texts using logical recursion boundary delimiters
splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)
chunks = splitter.split_text(text)
```

Step 3 & 4: Store Vector Database & Query Pipeline

```python
from langchain_community.embeddings import OpenAIEmbeddings
from langchain_community.vectorstores import Chroma
from langchain.chains import RetrievalQA
from langchain_community.chat_models import ChatOpenAI

# Generate embeddings and initialize local DB
db = Chroma.from_texts(texts=chunks, embedding=OpenAIEmbeddings(), persist_directory="./db")

# Construct automatic retrieval generator chain
chain = RetrievalQA.from_chain_type(
    llm=ChatOpenAI(model="gpt-4o-mini", temperature=0),
    chain_type="stuff",
    retriever=db.as_retriever(search_kwargs={"k": 3})
)

answer = chain.run("What is the system baseline safety limit?")
```

3. Math & Core Metrics

Cosine Similarity Search Metric

Vector indices match user queries using mathematical orientation calculations:

$$\text{Cosine Similarity} = \cos(\theta) = (A \cdot B) / (\|A\| \times \|B\|)$$

Where A and B are the token coordinate arrays generated by your embedding model. Higher proximity values directly predict optimal text matches.